

Robótica
Informe ROS2 (robot operative system)

Docente: Jaime Enrique Arango Castro

Estudiante:
Cristhian Mateo Almeida Gómez

Universidad Nacional de Colombia
Sede Manizales



Instalación y Aprendizaje de ROS2 Humble

1. Instalación de Ubuntu 22.04 sobre WSL en Windows 11

El primer paso para trabajar con ROS2 Humble en un entorno Windows es instalar Ubuntu 22.04 sobre WSL (Windows Subsystem for Linux).

Este procedimiento permite usar Ubuntu como si fuera una máquina virtual, lo que facilita la ejecución de ROS2 sin la necesidad de un sistema operativo dual.

Pasos para la instalación:

1. Instalar WSL:

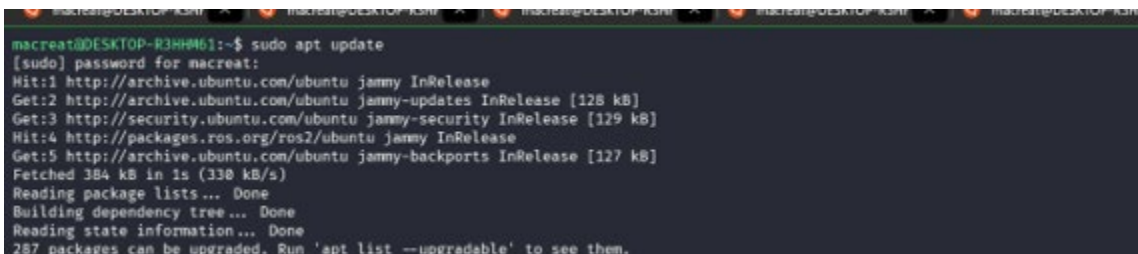
- Abrir PowerShell como administrador y ejecutar:

```
wsl --install
```

Esto instalará WSL y procederemos a descargar UBUNTU o la versión reciente que deseemos.

2. Actualizar el sistema:

- Una vez dentro de nuestro sistema UBUNTU en WSL (WINDOWS SubSYSTEM FOR LINUX) Ejecutar los comandos para asegurarse de que todo el sistema esté actualizado:



```
macreat@DESKTOP-R3HHM61:~$ sudo apt update
[sudo] password for macreat:
Hit:1 http://archive.ubuntu.com/ubuntu jammy InRelease
Get:2 http://archive.ubuntu.com/ubuntu jammy-updates InRelease [128 kB]
Get:3 http://security.ubuntu.com/ubuntu jammy-security InRelease [129 kB]
Hit:4 http://packages.ros.org/ros2/ubuntu jammy InRelease
Get:5 http://archive.ubuntu.com/ubuntu jammy-backports InRelease [127 kB]
Fetched 384 kB in 1s (330 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
287 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

También sudo apt upgrade

2. Instalación de ROS2 Humble

Para instalar ROS2 Humble en Ubuntu 22.04 bajo WSL, realizamos el siguiente trabajo:

1. Configuración de repositorios de ROS2:

- Agregar las claves y repositorios de ROS2:

```
curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.asc | sudo tee  
/etc/apt/trusted.gpg.d/ros.asc  
sudo sh -c 'echo "deb [arch=amd64] http://packages.ros.org/ros2/ubuntu focal main" >  
/etc/apt/sources.list.d/ros2.list'
```

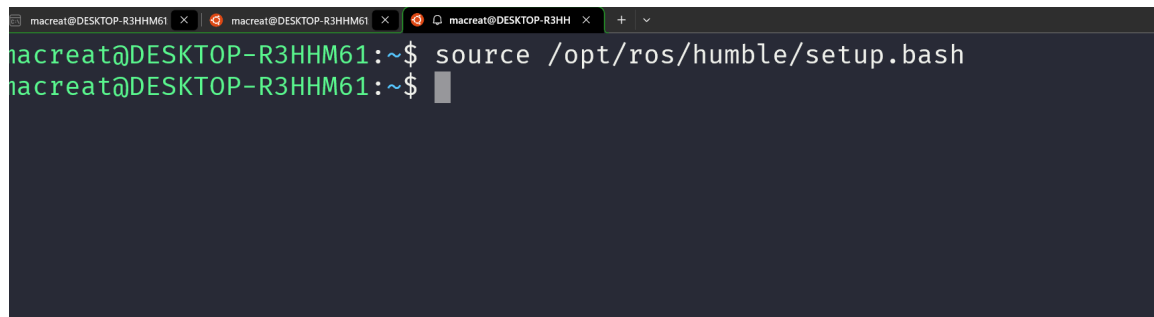
2. Instalación de ROS2 Humble:

- Instalar la versión completa de ROS2 Humble:

```
sudo apt update  
sudo apt install ros-humble-desktop
```

3. Configurar el entorno:

- Agregar ROS2 al entorno de ejecución de bash:

A screenshot of a terminal window with three tabs. The active tab shows the command `source /opt/ros/humble/setup.bash` being entered at the prompt `macreat@DESKTOP-R3HHM61:~$`. The prompt changes to `macreat@DESKTOP-R3HHM61:~$` after the command is executed.

```
macreat@DESKTOP-R3HHM61:~$ source /opt/ros/humble/setup.bash  
macreat@DESKTOP-R3HHM61:~$
```

4. Verificación de la instalación:

- Verificar que ROS2 se haya instalado correctamente ejecutando es crucial para continuar con un entorno de desarrollo correcto:

```

macreat@DESKTOP-R3HHM61:~$ echo $ROS_DISTRO
humble
macreat@DESKTOP-R3HHM61:~$ ls /opt/ros/humble
bin      lib          local_setup.sh      opt      setup.zsh  tools
cmake    local        _local_setup_util.py  setup.bash  share
include  local_setup.bash  local_setup.zsh      setup.sh   src
macreat@DESKTOP-R3HHM61:~$ █

```

3. Preparación del Informe sobre los Tutoriales de ROS2

3.1 CLI Tools

Los comandos de la CLI de ROS2 permiten interactuar con el sistema ROS sin necesidad de escribir código, lo que facilita la gestión de nodos, tópicos, servicios y más.

Veamos una verificación una vez estamos en nuestro espacio de Ubuntu con las dependencias instaladas:

```

macreat@DESKTOP-R3HHM61:~$ ros2 topic list
/parameter_events
/rosout

```

1. Configuración del entorno:

- Asegurarse de que ROS2 Humble está correctamente instalado y “sourced” con:
source /opt/ros/humble/setup.bash
- Verificar que el entorno esté listo utilizando:
ros2 --help

2. Comprobación de nodos y tópicos:

- Para ver los nodos activos en el sistema:
ros2 node list
- Para ver los tópicos activos:
ros2 topic list

Al comienzo al listar los nodos o tópicos, están vacíos ::

```
macreat@DESKTOP-R3HHM61:~$ ros2 node list
/minimal_service
macreat@DESKTOP-R3HHM61:~$
```

3. Uso de Turtlesim:

- Turtlesim es un simulador básico en ROS que permite experimentar con conceptos fundamentales de ROS2, como la creación y suscripción de tópicos:

```
ros2 run turtlesim turtlesim_node
ros2 topic list
```

4. Uso de rqt:

- rqt es una herramienta gráfica que permite interactuar con ROS visualmente, como invocar servicios y monitorear los tópicos:

```
ros2 run rqt_gui rqt_gui
```

Procedamos con su instalación:

```
macreat@DESKTOP-R3HHM61:~$ sudo apt update
[sudo] password for macreat:
Hit:1 http://archive.ubuntu.com/ubuntu jammy InRelease
Get:2 http://archive.ubuntu.com/ubuntu jammy-updates InRelease [128 kB]
Get:3 http://security.ubuntu.com/ubuntu jammy-security InRelease [129 kB]
Hit:4 http://packages.ros.org/ros2/ubuntu jammy InRelease
Get:5 http://archive.ubuntu.com/ubuntu jammy-backports InRelease [127 kB]
Fetched 384 kB in 1s (330 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
287 packages can be upgraded. Run 'apt list --upgradable' to see them.
macreat@DESKTOP-R3HHM61:~$ sudo apt install ros-humble-turtlesim
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following packages will be upgraded:
  ros-humble-turtlesim
1 upgraded, 0 newly installed, 0 to remove and 286 not upgraded.
Need to get 497 kB of archives.
After this operation, 0 B of additional disk space will be used.
Get:1 http://packages.ros.org/ros2/ubuntu jammy/main amd64 ros-humble-turtlesim amd64 1.4.2-1jammy.20250617.224330 [497 kB]
Fetched 497 kB in 1s (535 kB/s)
(Reading database ... 136996 files and directories currently installed.)
Preparing to unpack .../ros-humble-turtlesim_1.4.2-1jammy.20250617.224330_amd64.deb ...
Unpacking ros-humble-turtlesim (1.4.2-1jammy.20250617.224330) over (1.4.2-1jammy.20250429.211545) ...
Setting up ros-humble-turtlesim (1.4.2-1jammy.20250617.224330) ...
Processing triggers for libc-bin (2.35-0ubuntu3.10) ...
macreat@DESKTOP-R3HHM61:~$ sudo apt install 'ros-humble-rqt*'
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Note, selecting 'ros-humble-rqt-mocap4r2-control-dbgSYM' for glob 'ros-humble-rqt*'
Note, selecting 'ros-humble-rqt-robot-dashboard' for glob 'ros-humble-rqt*'
Note, selecting 'ros-humble-rqt-runtime-monitor' for glob 'ros-humble-rqt*'
Note, selecting 'ros-humble-rqt-gui-py' for glob 'ros-humble-rqt*'
Note, selecting 'ros-humble-rqt-console' for glob 'ros-humble-rqt*'
Note, selecting 'ros-humble-rqt-shell' for glob 'ros-humble-rqt*'
Note, selecting 'ros-humble-rqt-reconfigure' for glob 'ros-humble-rqt*'
Note, selecting 'ros-humble-rqt-py-console' for glob 'ros-humble-rqt*'
Note, selecting 'ros-humble-rqt-robot-monitor' for glob 'ros-humble-rqt*'
Note, selecting 'ros-humble-rqt-publisher' for glob 'ros-humble-rqt*'
Note, selecting 'ros-humble-rqt-robot-steering' for glob 'ros-humble-rqt*'
Note, selecting 'ros-humble-rqt-image-overlay-dbgSYM' for glob 'ros-humble-rqt*'
Note, selecting 'ros-humble-rqt-image-overlay-layer' for glob 'ros-humble-rqt*'
Note, selecting 'ros-humble-rqt-image-view-dbgSYM' for glob 'ros-humble-rqt*'
Note, selecting 'ros-humble-rqt-common-plugins' for glob 'ros-humble-rqt*'
Note, selecting 'ros-humble-rqt-topic' for glob 'ros-humble-rqt*'
```

podemos correr nuestros nodos desde la respectiva capa:

```
etting up ros-humble-rqt-runtime-monitor (1.0.0-3jammy.20250617.211902) ...
etting up ros-humble-rqt-mocap4r2-control (0.0.7-1jammy.20250617.223447) ...
etting up ros-humble-rqt-publisher (1.5.0-1jammy.20250607.093139) ...
etting up ros-humble-rqt-image-ov (1.5.0-1jammy.20250617.225941) ...
etting up ros-humble-rqt-msg (1.2.0-1jammy.20250617.221223) ...
etting up ros-humble-rqt-robot-da (1.0.0-1jammy.20250617.221223) ...
etting up ros-humble-rqt-srv (1.0.0-1jammy.20250617.221223) ...
etting up ros-humble-backward-ros (1.0.0-1jammy.20250617.221223) ...
etting up ros-humble-rqt-bag-plug (1.0.0-1jammy.20250617.221223) ...
etting up ros-humble-rqt-mocap4r2 (1.0.0-1jammy.20250617.223447) ...
etting up ros-humble-rqt-action (1.0.0-1jammy.20250617.223447) ...
etting up ros-humble-controller-m (1.0.0-1jammy.20250617.225851) ...
etting up ros-humble-rqt-controll (1.0.0-1jammy.20250617.234645) ...
etting up ros-humble-rqt-common-p (1.0.0-1jammy.20250618.005953) ...
rocessing triggers for man-db (2.11.2-1ubuntu1.1) ...
rocessing triggers for libc-bin (2.35-0ubuntu3.10) ...
acreat@DESKTOP-R3HHM61:~$ source /opt/ros/humble/setup.bash
acreat@DESKTOP-R3HHM61:~$ ros2 run turtlesim turtlesim_node
StandardPaths: wrong permissions on runtime directory /run/user/1000/, 0755 instead of 000
INFO [1750772512.118337696] [turtlesim]: Starting turtlesim with node name /turtlesim
INFO [1750772512.151592683] [turtlesim]: Spawning turtle [turtle1] at x=[5.544445], y=[5.544445], theta=[0.000000]
```

Y de esta forma, podemos en otra terminal como overlay, usar turtle teleop key, para manejar nuestra tortuga en la underlay:

```
macreat@DESKTOP-R3HHM61:~$ source /opt/ros/humble/setup.bash
macreat@DESKTOP-R3HHM61:~$ ros2 run turtlesim turtle_teleop_key
Reading from keyboard
Use arrow keys to move the turtle.
Use G|B|V|C|D|E|R|T keys to rotate to absolute orientations. 'F' to cancel a rotation
'Q' to quit.
```

3.2 Client Libraries

Las Client Libraries de ROS2 permiten que los nodos interactúen con el sistema ROS mediante código en Python o C++. En este caso, nos enfocamos en el uso de bibliotecas para crear nodos de cliente y servicio.

1. Creación de un Workspace:

- Se crea un nuevo espacio de trabajo para desarrollar paquetes de ROS2:

- Enrutamos nuestra fuente:
 - Source /opt/ros/humble/setup.bash
- Creamos un directorio
 - Mkdir...
- Clonamos el repositorio en la carpeta fuente (src)
 - Usamos un paquete ya existente:
 - Un repositorio puede tener múltiples ramas. Debes verificar cuál de ellas está dirigida a la distribución de **ROS 2** que tienes instalada. Cuando clones este repositorio, agrega el argumento -b seguido de la rama correspondiente.
 - Procedemos a clonar el repositorio en la rama humble:

```
macreat@DESKTOP-R3HHM61:~/ros2_ws/src$ git clone https://github.com/ros/ros_tutorials.git -b humble
Cloning into 'ros_tutorials'...
remote: Enumerating objects: 3179, done.
remote: Counting objects: 100% (133/133), done.
remote: Compressing objects: 100% (57/57), done.
remote: Total 3179 (delta 93), reused 84 (delta 76), pack-reused 3046 (from 3)
Receiving objects: 100% (3179/3179), 751.30 KiB | 2.45 MiB/s, done.
Resolving deltas: 100% (1909/1909), done.
macreat@DESKTOP-R3HHM61:~/ros2_ws/src$
```

- Resolución de dependencias
 - Es útil resolver las dependencias antes de construir el espacio de trabajo, ya que no querrás enfrentar problemas por dependencias faltantes después de ejecutar la construcción con **colcon**.

```
macreat@DESKTOP-R3HHM61:~/ros2_ws$ rosdep install -i --from-path src --rosdistro humble -y
#All required rosdeps installed successfully
macreat@DESKTOP-R3HHM61:~/ros2_ws$
```

- Es importante señalar que los paquetes declaran sus dependencias en el archivo **package.xml**. Este archivo contiene toda la información sobre las bibliotecas o paquetes que necesita el paquete para funcionar correctamente.

- Construcción del espacio de trabajo con COLCON:

Una vez que hayas instalado **colcon**, puedes proceder a construir tu espacio de trabajo. Para esto, debes ejecutar **colcon build** en la raíz de tu espacio de trabajo **ROS2**:

```
macreat@DESKTOP-R3HHM61:~$ sudo apt install python3-colcon-common-extensions
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
python3-colcon-common-extensions is already the newest version (0.3.0-100).
0 upgraded, 0 newly installed, 0 to remove and 286 not upgraded.
macreat@DESKTOP-R3HHM61:~$
```

Otros argumentos ÚTILES para colcon BUILD:

- `--packages-up-to` : Este argumento permite construir el paquete que desees, junto con todas sus dependencias, pero no todo el espacio de trabajo. Esto ahorra tiempo al evitar la reconstrucción de paquetes que no han sido modificados.
- `--symlink-install` : Usar este argumento permite que los archivos modificados, como los scripts de Python, se reflejen sin necesidad de reconstruir todo el paquete cada vez. Esto es útil cuando se realizan ajustes en el código y se desea que los cambios sean reflejados inmediatamente.
- `--event-handlers console_direct+` : Este argumento muestra la salida de la consola durante el proceso de construcción. De lo contrario, la salida se guarda en el directorio de logs, lo que puede ser útil para monitorear el progreso en tiempo real.
- `Colcon build --executor sequential` : Este argumento procesa los paquetes de manera secuencial, uno por uno, en lugar de utilizar paralelismo. Esto puede ser útil cuando se necesitan evitar problemas relacionados con la ejecución en

paralelo, aunque generalmente la construcción paralela es más rápida.

```
macreat@DESKTOP-R3HHM61:~/ros2_ws$ colcon build
[0.781s] WARNING:colcon.colcon_core.package_selection:Some selected packages are already built in one or more
underlay workspaces:
'turtlesim' is in: /opt/ros/humble
If a package in a merged underlay workspace is overridden and it installs headers, then all packages in the ov
erlay must sort their include directories by workspace order. Failure to do so may result in build failures or
undefined behavior at run time.
If the overridden package is used by another package in any underlay, then the overriding package in the overl
ay must be API and ABI compatible or undefined behavior at run time may occur.

If you understand the risks and want to override a package anyways, add the following to the command line:
--allow-overriding turtlesim

This may be promoted to an error in a future release of colcon-override-check.
Starting >>> turtlesim
[Processing: turtlesim]
Finished <<< turtlesim [53.2s]

Summary: 1 package finished [53.6s]
macreat@DESKTOP-R3HHM61:~/ros2_ws$ ls
build  install  log  src
macreat@DESKTOP-R3HHM61:~/ros2_ws$ cd src
macreat@DESKTOP-R3HHM61:~/ros2_ws/src$ ls
ros_tutorials
macreat@DESKTOP-R3HHM61:~/ros2_ws/src$ cd ..
macreat@DESKTOP-R3HHM61:~/ros2_ws$ ls install
COLCON_IGNORE  local_setup.ps1  _local_setup_util_ps1.py  local_setup.zsh  setup.ps1  setup.zsh
local_setup.bash  local_setup.sh  _local_setup_util_sh.py  setup.bash  setup.sh  turtlesim
macreat@DESKTOP-R3HHM61:~/ros2_ws$
```

El **directorio de instalación** es donde se encuentran los archivos de configuración de tu espacio de trabajo (workspace, WS). Estos archivos se utilizan para **source** (cargar) el **overlay** de tu espacio de trabajo.

- Sourcing del OVERLAY

Fuente del entorno principal de ROS2:

- En una nueva terminal, primero debes **source** el entorno principal de **ROS2** (denominado **underlay**), para que puedas construir y trabajar sobre tu espacio de trabajo (overlay):

```
macreat@DESKTOP-R3HHM61:~$ source /opt/ros/humble/setup.bash
macreat@DESKTOP-R3HHM61:~$
```

Este paso carga el entorno principal de **ROS2 Humble**, proporcionando acceso a las herramientas y bibliotecas esenciales de ROS2.

Fuente del espacio de trabajo (Overlay):

- Una vez que el entorno principal de **ROS2** está cargado, debes **source** el archivo de configuración de tu espacio de trabajo (overlay) para que tu entorno de trabajo se construya sobre el entorno principal:

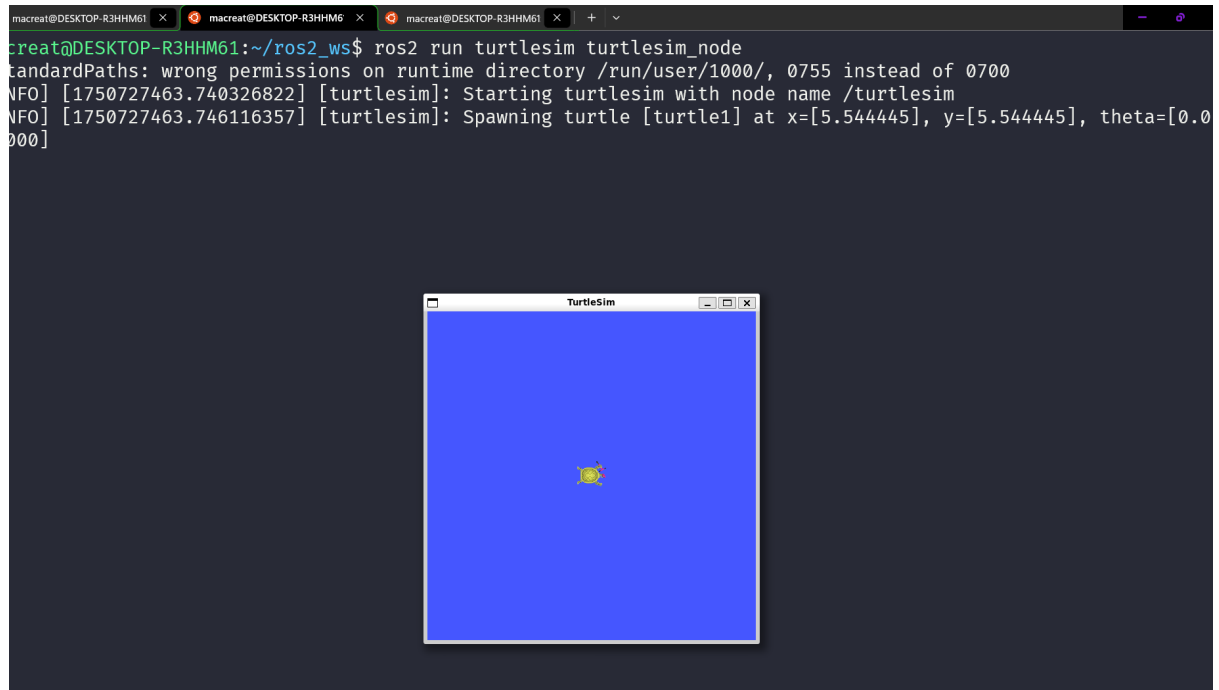
```
macreat@DESKTOP-R3HHM61:~$ source /opt/ros/humble/setup.bash
macreat@DESKTOP-R3HHM61:~$ cd ~/ros2_es
-bash: cd: /home/macreat/ros2_es: No such file or directory
macreat@DESKTOP-R3HHM61:~$ cd ~/ros2_ws
macreat@DESKTOP-R3HHM61:~/ros2_ws$ source install/local_setup.bash
macreat@DESKTOP-R3HHM61:~/ros2_ws$ ls
build  install  log  src
macreat@DESKTOP-R3HHM61:~/ros2_ws$ ls install
COLCON_IGNORE  local_setup.sh  local_setup.zsh  setup.sh
local_setup.bash  _local_setup_util_ps1.py  setup.bash  setup.zsh
local_setup.ps1  _local_setup_util_sh.py  setup.ps1  turtlesim
macreat@DESKTOP-R3HHM61:~/ros2_ws$
```

Esto asegura que los paquetes y configuraciones de tu espacio de trabajo estén disponibles y se integren correctamente con el entorno de **ROS2**.

El proceso de **source** tanto del entorno principal de **ROS2** como del espacio de trabajo (overlay) es equivalente a hacer solo **source** del archivo `setup.bash` de tu espacio de trabajo, ya que este archivo ya incluye el entorno de su **underlay** (el entorno principal de **ROS2**).

Ejecutar el Paquete Turtlesim desde el Overlay

Una vez que el overlay ha sido configurado y compilado, puedes ejecutar el paquete Turtlesim directamente desde tu espacio de trabajo ROS2. Esto te permitirá interactuar con el simulador y ver los resultados en la pantalla.



El código fuente de los nodos, como el de **Turtlesim**, se encuentra en el directorio de origen (src) dentro del paquete correspondiente en el espacio de trabajo. Para modificar el comportamiento del nodo, accede al archivo **.cpp** y realiza los cambios necesarios en la lógica del nodo, luego se compila el paquete, enruta la fuente y ejecutamos el nodo.

2. Creación de un Paquete en Python

Para crear un **paquete de Python en ROS2**, se utiliza el siguiente comando:

```
bash
```

CopiarEditar

```
ros2 pkg create --build-type ament_python --license Apache-2.0 my_package
```

Este paso es útil para compartir o instalar tu código en otro espacio de trabajo.

Estructura de un Paquete en ROS2

Paquetes CMake (C++):

- **CMakeLists.txt**: Archivo de configuración para construir el código dentro del paquete.
- **include/<package_name>**: Directorio que contiene los encabezados públicos del paquete.
- **package.xml**: Archivo con la información meta del paquete.
- **src**: Directorio que contiene el código fuente del paquete.

Paquetes Python:

- **package.xml**: Archivo que contiene la información meta del paquete.
- **resource/<package_name>**: Archivo de marcador para el paquete.
- **setup.cfg**: Necesario cuando un paquete tiene ejecutables, para que ros2 run pueda encontrarlos.
- **setup.py**: Instrucciones para instalar el paquete.
- **<package_name>**: Directorio con el mismo nombre que tu paquete, utilizado por las herramientas de ROS2 para encontrar el paquete. Contiene el archivo `__init__.py`.

Estructura recomendada para un paquete en Python:

plaintext

CopiarEditar

my_package/

```
|— package.xml
|— resource/my_package
|— setup.cfg
|— setup.py
└— my_package/
```

└── __init__.py

Práctica recomendada:

Es recomendable tener un directorio **src** dentro de tu espacio de trabajo, y crear los paquetes dentro de él. Esto mantiene el nivel superior del espacio de trabajo “limpio” y organizado.

Creación del Paquete en ROS2

En la raíz del directorio src:

Para crear un nuevo paquete en **ROS2**, se deben utilizar los siguientes comandos según el tipo de paquete:

ON C MAKE :

```
ros2 pkg create --build-type ament_cmake --license Apache-2.0 <package_name>
```

ON PY :

```
ros2 pkg create --build-type ament_python --license Apache-2.0 <package_name>
```

```
macreat@DESKTOP-R3HHM61:~/ros2_ws/src$ ros2 pkg create --build-type ament_python --license Apache-2.0 myPyPackage
going to create a new package
package name: myPyPackage
destination directory: /home/macreat/ros2_ws/src
package format: 3
version: 0.0.0
description: TODO: Package description
maintainer: ['macreat <macreat@todo.todo>']
licenses: ['Apache-2.0']
build type: ament_python
dependencies: []
creating folder ./myPyPackage
creating ./myPyPackage/package.xml
creating source folder
creating folder ./myPyPackage/myPyPackage
creating ./myPyPackage/setup.py
creating ./myPyPackage/setup.cfg
creating folder ./myPyPackage/resource
creating ./myPyPackage/resource/myPyPackage
creating ./myPyPackage/myPyPackage/__init__.py
creating folder ./myPyPackage/test
creating ./myPyPackage/test/test_copyright.py
creating ./myPyPackage/test/test_flake8.py
creating ./myPyPackage/test/test_pep257.py
macreat@DESKTOP-R3HHM61:~/ros2_ws/src$
```

Luego usamos :

Después de crear el paquete, se puede usar el argumento `--node-name`, que genera un ejecutable de tipo "Hello World" en el paquete:

```
macreat@DESKTOP-R3HHM61:~/ros2_ws/src$ ros2 pkg create --build-type ament_python --license Apache-2.0 --node-name myNode myPyPackageDir
going to create a new package
package name: myPyPackageDir
destination directory: /home/macreat/ros2_ws/src
package format: 3
version: 0.0.0
description: TODO: Package description
maintainer: ['macreat <macreat@todo.todo>']
licenses: ['Apache-2.0']
build type: ament_python
dependencies: []
node_name: myNode
creating folder ./myPyPackageDir
creating ./myPyPackageDir/package.xml
creating source folder
creating folder ./myPyPackageDir/myPyPackageDir
creating ./myPyPackageDir/setup.py
creating ./myPyPackageDir/setup.cfg
creating folder ./myPyPackageDir/resource
creating ./myPyPackageDir/resource/myPyPackageDir
creating ./myPyPackageDir/myPyPackageDir/__init__.py
creating folder ./myPyPackageDir/test
creating ./myPyPackageDir/test/test_copyright.py
creating ./myPyPackageDir/test/test_flake8.py
creating ./myPyPackageDir/test/test_pep257.py
creating ./myPyPackageDir/myPyPackageDir/myNode.py
macreat@DESKTOP-R3HHM61:~/ros2_ws/src$ ls
a.out  myPyPackage  myPyPackageDir  ros_tutorials
macreat@DESKTOP-R3HHM61:~/ros2_ws/src$
```

Construcción del Paquete

Una vez creado el paquete, es necesario construirlo en el espacio de trabajo. Para ello, en la raíz del espacio de trabajo (WS), ejecuta el siguiente comando:

```
macreat@DESKTOP-R3HHM61:~/ros2_ws$ colcon build
WARNING: Package name "myPyPackage" does not follow the naming conventions. It should start with a lower case letter and only contain lowercase letters, digits, underscores, and dashes.
WARNING: Package name "myPyPackageDir" does not follow the naming conventions. It should start with a lower case letter and only contain lowercase letters, digits, underscores, and dashes.
[0.734s] WARNING:colcon.colcon_core.package_selection:Some selected packages are already built in one or more underlay workspaces:
'turtlesim' is in: /home/macreat/ros2_ws/install/turtlesim, /opt/ros/humble
If a package in a merged underlay workspace is overridden and it installs headers, then all packages in the overlay must sort their include directories by workspace order. Failure to do so may result in build failures or undefined behavior at run time.
If the overridden package is used by another package in any underlay, then the overriding package in the overlay must be API and ABI compatible or undefined behavior at run time may occur.

If you understand the risks and want to override a package anyways, add the following to the command line:
--allow-overriding turtlesim

This may be promoted to an error in a future release of colcon-override-check.
Starting >>> myPyPackage
Starting >>> myPyPackageDir
Starting >>> turtlesim
Finished <<< turtlesim [1.68s]
Finished <<< myPyPackageDir [1.92s]
Finished <<< myPyPackage [1.95s]

Summary: 3 packages finished [2.41s]
macreat@DESKTOP-R3HHM61:~/ros2_ws$ colcon build --packages-select myPyPackage
WARNING: Package name "myPyPackage" does not follow the naming conventions. It should start with a lower case letter and only contain lowercase letters, digits, underscores, and dashes.
WARNING: Package name "myPyPackageDir" does not follow the naming conventions. It should start with a lower case letter and only contain lowercase letters, digits, underscores, and dashes.
Starting >>> myPyPackage
Finished <<< myPyPackage [0.88s]

Summary: 1 package finished [1.15s]
```

Este comando permite seleccionar únicamente el paquete **my_package** para su construcción e instalación, evitando que todo el espacio de trabajo sea recompilado. Esto es útil cuando solo se desea compilar un paquete específico en el espacio de trabajo.

Después de esto, puedo enrutar la fuente desde el `setup.bash`

```
macreat@DESKTOP-R3HHM61 x macreat@DESKTOP-R3HHM6 x + v
macreat@DESKTOP-R3HHM61:~/ros2_ws$ source install/local_setup.bash
```

Y luego usar ‘myNode’ personalizado creado en ‘myPyPackageDir’

```
macreat@DESKTOP-R3HHM61 x macreat@DESKTOP-R3HHM6 x + v
macreat@DESKTOP-R3HHM61:~/ros2_ws$ ros2 run myPyPackageDir myNode
Hi from myPyPackageDir.
macreat@DESKTOP-R3HHM61:~/ros2_ws$
```

Podemos ver el contenido del paquete:

```
macreat@DESKTOP-R3HHM61:~/ros2_ws/src$ cd myPyPackageDir
macreat@DESKTOP-R3HHM61:~/ros2_ws/src/myPyPackageDir$ ls
LICENSE  myPyPackageDir  package.xml  resource  setup.cfg  setup.py  test
macreat@DESKTOP-R3HHM61:~/ros2_ws/src/myPyPackageDir$
```

Personalización del Paquete en package.xml

Para personalizar el contenido del paquete, es necesario editar el archivo **package.xml**. A continuación, se detallan los pasos para ajustar la información del mantenedor, el correo electrónico del mantenedor y la descripción del paquete:

1. Editar package.xml:

- Modificar las siguientes líneas en **package.xml** para que coincidan con los datos personalizados del paquete:

```
<maintainer email="your_email@example.com">Your Name</maintainer>
```

```
<description>A brief description of the package</description>
```

2. Guardar el archivo:

- Después de realizar las modificaciones necesarias, guarda el archivo **package.xml** con la extensión **.xml**.

3. Edición de setup.py (Solo en Paquetes Python):

- En paquetes de **Python**, también es necesario editar el archivo **setup.py** para indicar cómo instalar el paquete. Este paso no es necesario en paquetes **CMake**.

3. Publicador y Suscriptor:

En **ROS2**, los nodos se comunican mediante **tópicos**. Un nodo **publicador** envía datos a un tópico, mientras que un nodo **suscriptor** recibe esos datos.

La idea es crear nodos que se comuniquen entre sí enviando mensajes de tipo **string** a través de un tópico. Este sistema funciona como un **"talker"** y **"listener"**, donde un nodo publica los datos y el otro se suscribe al tópico para recibirlos.

- Creamos el paquete desde la carpeta/directorio fuente (SRC)

```
macreat@DESKTOP-R3HHM61:~/ros2_ws$ cd src
macreat@DESKTOP-R3HHM61:~/ros2_ws/src$ ros2 pkg create --build-type ament_python --license Apache-2.0 py_pubsub
going to create a new package
package name: py_pubsub
destination directory: /home/macreat/ros2_ws/src
package format: 3
version: 0.0.0
description: TODO: Package description
maintainer: ['macreat <macreat@todo.todo>']
licenses: ['Apache-2.0']
build type: ament_python
dependencies: []
creating folder ./py_pubsub
creating ./py_pubsub/package.xml
creating source folder
creating folder ./py_pubsub/py_pubsub
creating ./py_pubsub/setup.py
creating ./py_pubsub/setup.cfg
creating folder ./py_pubsub/resource
creating ./py_pubsub/resource/py_pubsub
creating ./py_pubsub/py_pubsub/__init__.py
creating folder ./py_pubsub/test
creating ./py_pubsub/test/test_copyright.py
creating ./py_pubsub/test/test_flake8.py
creating ./py_pubsub/test/test_pep257.py
macreat@DESKTOP-R3HHM61:~/ros2_ws/src$
```

- Escribimos el nodo publicador:

Dentro del paquete, se puede descargar el ejemplo de **Talker** desde el contenido del repositorio de Git. Esto añadirá un nuevo archivo llamado **publisher_member_function.py**, que estará ubicado junto al archivo **__init__.py** dentro del espacio de trabajo del paquete.

```
macreat@DESKTOP-R3HHM61:~/ros2_ws/src$ ls
a.out myPyPackageDir py_pubsub ros_tutorials
macreat@DESKTOP-R3HHM61:~/ros2_ws/src$ cd py_pubsub
macreat@DESKTOP-R3HHM61:~/ros2_ws/src/py_pubsub$ cd py_pubsub
macreat@DESKTOP-R3HHM61:~/ros2_ws/src/py_pubsub/py_pubsub$ wget https://raw.githubusercontent.com/ros2/examples/humble/rclpy/topics/minimal_publisher/examples_rclpy_minimal_publisher/publisher_member_function.py
--2025-06-23 21:11:28-- https://raw.githubusercontent.com/ros2/examples/humble/rclpy/topics/minimal_publisher/examples_rclpy_minimal_publisher/publisher_member_function.py
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.111.133, 185.199.109.133, 185.199.108.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.111.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 1576 (1.5K) [text/plain]
Saving to: 'publisher_member_function.py'

publisher_member_function.py      100%[=====>] 1.54K --.-KB/s  in 0s

2025-06-23 21:11:29 (24.6 MB/s) - 'publisher_member_function.py' saved [1576/1576]
macreat@DESKTOP-R3HHM61:~/ros2_ws/src/py_pubsub/py_pubsub$
```

Este archivo **publisher_member_function.py** contiene la implementación de un nodo **publicador** que permite la comunicación en ROS2 mediante tópicos, enviando mensajes de tipo **string** a otros nodos suscriptores.

```

publisher_member_function.py          100%[=====]
2025-06-23 21:11:29 (24.6 MB/s) - 'publisher_member_function.py' saved [1576/1576]

macreat@DESKTOP-R3HHM61:~/ros2_ws/src/py_pubsub/py_pubsub$ ls
__init__.py  publisher_member_function.py
macreat@DESKTOP-R3HHM61:~/ros2_ws/src/py_pubsub/py_pubsub$ █

```

Podemos abrir el archivo usando el método NANO, y ver su contenido directamente.

```

GNU nano 6.2                                     publisher_member_function.py
# See the License for the specific language governing permissions and
# limitations under the License.

import rclpy
from rclpy.node import Node

from std_msgs.msg import String

class MinimalPublisher(Node):

    def __init__(self):
        super().__init__('minimal_publisher')
        self.publisher_ = self.create_publisher(String, 'topic', 10)
        timer_period = 0.5 # seconds
        self.timer = self.create_timer(timer_period, self.timer_callback)
        self.i = 0

    def timer_callback(self):
        msg = String()
        msg.data = 'Hello World: %d' % self.i
        self.publisher_.publish(msg)
        self.get_logger().info('Publishing: "%s"' % msg.data)
        self.i += 1

def main(args=None):
    rclpy.init(args=args)

    minimal_publisher = MinimalPublisher()

    rclpy.spin(minimal_publisher)

^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Locat
^X Exit      ^R Read File  ^_ Replace    ^U Paste      ^J Justify    ^/ Go To

```

```

def __init__(self):
    super().__init__('minimal_publisher')
    self.publisher_ = self.create_publisher(String, 'topic', 10)
    timer_period = 0.5 # seconds
    self.timer = self.create_timer(timer_period, self.timer_callback)
    self.i = 0

def timer_callback(self):
    msg = String()
    msg.data = 'Hello World: %d' % self.i
    self.publisher_.publish(msg)
    self.get_logger().info('Publishing: "%s"' % msg.data)
    self.i += 1

def main(args=None):
    rclpy.init(args=args)

    minimal_publisher = MinimalPublisher()

    rclpy.spin(minimal_publisher)

    # Destroy the node explicitly
    # (optional - otherwise it will be done automatically
    # when the garbage collector destroys the node object)
    minimal_publisher.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

 Help	 Write Out	 Where Is	 Cut	 Execute
 Exit	 Read File	 Replace	 Paste	 Justify

- Podemos proceder añadiendo las dependencias.

Vamos al Directorio de mi paquete y abrimos el archivo package.xml

```

acreat@DESKTOP-R3HHM61:~/ros2_ws/src/py_pubsub/py_pubsub$ cd ..
acreat@DESKTOP-R3HHM61:~/ros2_ws/src/py_pubsub$ ls
LICENSE package.xml py_pubsub resource setup.cfg setup.py test
acreat@DESKTOP-R3HHM61:~/ros2_ws/src/py_pubsub$ nano package.xml
acreat@DESKTOP-R3HHM61:~/ros2_ws/src/py_pubsub$

```

```

<name>py_pubsub</name>
<version>0.0.0</version>
<description>examples of minimal publisher/subscriber using rclpy</description>
<maintainer email="macreat@todo.todo">macreat</maintainer>
<license>Apache-2.0</license>

<exec_depend>rclpy</exec_depend>
<exec_depend>std_msgs</exec_depend>

<test_depend>ament_copyright</test_depend>
<test_depend>ament_flake8</test_depend>

```

Agregar exec_depend y Punto de Entrada en setup.py

1. Agregar exec_depend:

Esto declara que el paquete necesita las dependencias **rclpy** y **std_msgs** cuando su código sea ejecutado. Esto asegura que las bibliotecas necesarias estén disponibles para el paquete en tiempo de ejecución.

2. Agregar un Punto de Entrada:

Abre el archivo **setup.py** y edita el archivo para agregar el punto de entrada del paquete. Esto es necesario para que **ROS2** pueda ejecutar el paquete de manera adecuada cuando se invoque desde la línea de comandos.

```

from setuptools import find_packages, setup

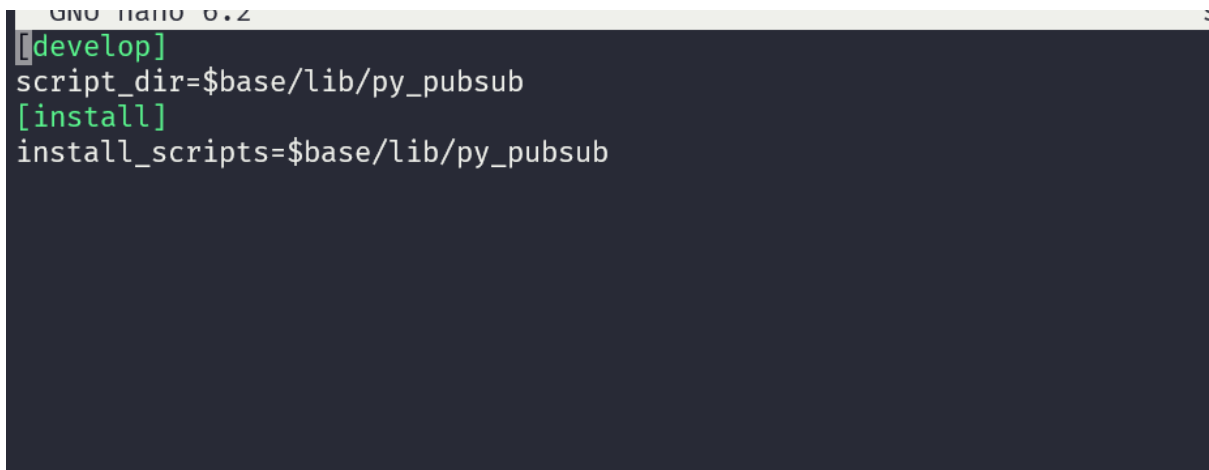
package_name = 'py_pubsub'

setup(
    name=package_name,
    version='0.0.0',
    packages=find_packages(exclude=['test']),
    data_files=[
        ('share/ament_index/resource_index/packages',
         ['resource/' + package_name]),
        ('share/' + package_name, ['package.xml']),
    ],
    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='macreat',
    maintainer_email='macreat@todo.todo',
    description='examples of minimal publisher/subscriber using rclpy',
    license='Apache-2.0',
    tests_require=['pytest'],
    entry_points={
        'console_scripts': [
            'talker = py_pubsub.publisher_member_function:main',
        ],
    },
)

```

Ponemos el talker' en los puntos de entrada (entry_points)

- Procedemos a revisar el archivo setup.cfg



```
GNU nano 0.2
[develop]
script_dir=$base/lib/py_pubsub
[install]
install_scripts=$base/lib/py_pubsub
```

La configuración del paquete se realiza utilizando **setuptools**, lo que permite indicar que los ejecutables del paquete se deben colocar en el directorio **lib**, ya que **ros2 run** buscará los ejecutables en ese directorio. Esta configuración asegura que el sistema de ejecución de **ROS2** pueda localizar y ejecutar los nodos de manera adecuada.

Una vez realizada esta configuración, se puede proceder con la construcción del paquete, hacer **source** de los archivos de configuración locales y ejecutar el paquete. No obstante, para observar el sistema en su totalidad y verificar su funcionamiento, se recomienda crear primero el nodo **suscriptor**

- Creación del NODO SUSCRIPTOR

Dentro del paquete, se debe crear o copiar el contenido necesario para implementar el nodo **suscriptor**. Una vez completado este paso, el directorio del paquete contendrá el archivo **subscriber_member_function.py**, que implementará la funcionalidad del nodo suscriptor.

```

macreat@DESKTOP-R3HHM61:~/ros2_ws/src/py_pubsub/py_pubsub$ ls
__init__.py publisher_member_function.py
macreat@DESKTOP-R3HHM61:~/ros2_ws/src/py_pubsub/py_pubsub$ wget https://raw.githubusercontent.com/ros2/examples/humble/rclpy/topics/minimal_subscriber/examples_rclpy_minimal_subscriber/subscriber_member_function.py
--2025-06-23 21:22:20-- https://raw.githubusercontent.com/ros2/examples/humble/rclpy/topics/minimal_subscriber/examples_rclpy_minimal_subscriber/subscriber_member_function.py
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.110.133, 185.199.111.133, 185.199.108.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.110.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 1469 (1.4K) [text/plain]
Saving to: 'subscriber_member_function.py'

subscriber_member_function.py      100%[=====>] 1.43K  --.-KB/s  in 0s

2025-06-23 21:22:20 (15.2 MB/s) - 'subscriber_member_function.py' saved [1469/1469]

macreat@DESKTOP-R3HHM61:~/ros2_ws/src/py_pubsub/py_pubsub$ ls
__init__.py publisher_member_function.py subscriber_member_function.py
macreat@DESKTOP-R3HHM61:~/ros2_ws/src/py_pubsub/py_pubsub$

```

Este nodo suscriptor será responsable de recibir los mensajes publicados en el tópico, permitiendo así la comunicación bidireccional en el sistema

Podemos abrir el archive para examinar su código:

```

class MinimalSubscriber(Node):

    def __init__(self):
        super().__init__('minimal_subscriber')
        self.subscription = self.create_subscription(
            String,
            'topic',
            self.listener_callback,
            10)
        self.subscription # prevent unused variable warning

    def listener_callback(self, msg):
        self.get_logger().info('I heard: "%s"' % msg.data)

def main(args=None):
    rclpy.init(args=args)

    minimal_subscriber = MinimalSubscriber()

    rclpy.spin(minimal_subscriber)

    # Destroy the node explicitly
    # (optional - otherwise it will be done automatically
    # when the garbage collector destroys the node object)
    minimal_subscriber.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

[^]G Help [^]O Write Out [^]W Where Is [^]K Cut
[^]X Exit [^]R Read File [^]\ Replace [^]U Paste

Dado que el nodo suscriptor tiene las mismas dependencias que el nodo publicador, no es necesario agregar nuevas dependencias al archivo **package.xml**. Además, el archivo **setup.cfg** también puede permanecer sin cambios, ya que no requiere modificaciones adicionales.

Para asegurar que el nodo suscriptor sea reconocido y ejecutado correctamente por **ROS2**, es necesario agregar un punto de entrada en el archivo **setup.py**. Este paso configura el nodo para que pueda ser ejecutado cuando se invoca desde la línea de comandos.

```
setup(
    name=package_name,
    version='0.0.0',
    packages=find_packages(exclude=['test']),
    data_files=[
        ('share/ament_index/resource_index/packages',
         ['resource/' + package_name]),
        ('share/' + package_name, ['package.xml']),
    ],
    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='macreat',
    maintainer_email='macreat@todo.todo',
    description='examples of minimal publisher/subscriber using rclpy',
    license='Apache-2.0',
    tests_require=['pytest'],
    entry_points={
        'console_scripts': [
            'talker = py_pubsub.publisher_member_function:main',
            'listener = py_pubsub.subscriber_member_function:main',
        ],
    },
)
```

- Procedemos compilando y corriendo nuestro paquete.

Una buena práctica es correr **ROSDEP** en la ruta de mi actual espacio de trabajo (**ros2_ws**) para revisar si hay dependencias perdidas u olvidadas antes de compilar.

```
rosdep install -i --from-path src --rosdistro humble -y
```

```

macreat@DESKTOP-R3HHM61:~/ros2_ws$ rosdep install -i --from-path src --rosdistro humble -y
WARNING: Package name "myPyPackageDir" does not follow the naming conventions. It should start with a lower case letter and only contain lower case letters, digits, underscores, and dashes.
WARNING: Package name "myPyPackageDir" does not follow the naming conventions. It should start with a lower case letter and only contain lower case letters, digits, underscores, and dashes.
WARNING: Package name "myPyPackageDir" does not follow the naming conventions. It should start with a lower case letter and only contain lower case letters, digits, underscores, and dashes.
#All required rosdeps installed successfully
macreat@DESKTOP-R3HHM61:~/ros2_ws$

```

En esta ruta, Podemos compilar nuestro paquete py_pub_sub

```

#All required rosdeps installed successfully
macreat@DESKTOP-R3HHM61:~/ros2_ws$ colcon build --packages-select py_pubsub
WARNING: Package name "myPyPackageDir" does not follow the naming conventions. It should start with a lower case letter and only contain lower case letters, digits, underscores, and dashes.
Starting >>> py_pubsub
Finished <<< py_pubsub [1.03s]

Summary: 1 package finished [1.56s]
macreat@DESKTOP-R3HHM61:~/ros2_ws$

```

Procedemos enrutando la fuente del entorno:

```

macreat@DESKTOP-R3HHM61:~$ ls
ros2_debs_backup  ros2_packages_cache  ros2ProofsWS  ros2_ws  ros2WS
macreat@DESKTOP-R3HHM61:~$ cd ros2_ws
macreat@DESKTOP-R3HHM61:~/ros2_ws$ ls
build  install  log  src
macreat@DESKTOP-R3HHM61:~/ros2_ws$ source install/setup.bash
macreat@DESKTOP-R3HHM61:~/ros2_ws$ ros2 run py_pubsub talker
[INFO] [1750732239.274961436] [minimal_publisher]: Publishing: "Hello World: 0"
[INFO] [1750732239.763391496] [minimal_publisher]: Publishing: "Hello World: 1"
[INFO] [1750732240.263407223] [minimal_publisher]: Publishing: "Hello World: 2"
[INFO] [1750732240.763825908] [minimal_publisher]: Publishing: "Hello World: 3"
[INFO] [1750732241.263391358] [minimal_publisher]: Publishing: "Hello World: 4"
[INFO] [1750732241.763373355] [minimal_publisher]: Publishing: "Hello World: 5"
[INFO] [1750732242.263526948] [minimal_publisher]: Publishing: "Hello World: 6"
[INFO] [1750732242.763702680] [minimal_publisher]: Publishing: "Hello World: 7"
[INFO] [1750732243.263403907] [minimal_publisher]: Publishing: "Hello World: 8"
[INFO] [1750732245.429861185] [minimal_publisher]: Publishing: "Hello World: 9"
[INFO] [1750732245.763831184] [minimal_publisher]: Publishing: "Hello World: 10"
[INFO] [1750732246.263601658] [minimal_publisher]: Publishing: "Hello World: 11"
[INFO] [1750732246.763600563] [minimal_publisher]: Publishing: "Hello World: 12"

```

En este entorno de desarrollo, se tiene un **overlay** y un **underlay**. El **overlay** se refiere al espacio de trabajo en el cual se realizan las modificaciones y donde se construyen los paquetes. El **underlay** es el entorno base donde residen las dependencias y herramientas fundamentales de **ROS2**, como el sistema de nodos y herramientas predeterminadas.

Ubicación del Listener en el Underlay

En este caso, el **Listener** (nodo suscriptor) se encuentra en el **underlay**, lo que significa que no modifica las dependencias ni la estructura del overlay. El Listener simplemente interactúa con el sistema a través de los mensajes enviados por el **Publisher** (nodo publicador) sin realizar cambios en el código del sistema base.

El nodo **Listener** se suscribe a los tópicos generados por el nodo **Publisher**, pero no afecta ni modifica las dependencias o configuraciones del entorno base (underlay). Este comportamiento refleja una de las ventajas de ROS2, donde el código y las funcionalidades adicionales se implementan y modifican en el overlay, sin alterar el funcionamiento del entorno subyacente.

```
macreat@DESKTOP-R3HHM61:~$ cd ros2_ws
macreat@DESKTOP-R3HHM61:~/ros2_ws$ ros2 run py_pubsub listener
Package 'py_pubsub' not found
macreat@DESKTOP-R3HHM61:~/ros2_ws$ ros2 run py_pubsub listener
Package 'py_pubsub' not found
macreat@DESKTOP-R3HHM61:~/ros2_ws$ source install/setup.bash
macreat@DESKTOP-R3HHM61:~/ros2_ws$ ros2 run py_pubsub listener
[INFO] [1750732326.275431867] [minimal_subscriber]: I heard: "Hello World: 165"
[INFO] [1750732326.763910396] [minimal_subscriber]: I heard: "Hello World: 166"
[INFO] [1750732327.264191814] [minimal_subscriber]: I heard: "Hello World: 167"
[INFO] [1750732327.763680585] [minimal_subscriber]: I heard: "Hello World: 168"
[INFO] [1750732328.264064057] [minimal_subscriber]: I heard: "Hello World: 169"
[INFO] [1750732328.764046271] [minimal_subscriber]: I heard: "Hello World: 170"
[INFO] [1750732329.263848367] [minimal_subscriber]: I heard: "Hello World: 171"
[INFO] [1750732329.763675215] [minimal_subscriber]: I heard: "Hello World: 172"
```

NOTEMOS QUE TAMBIÉN ES NECESARIO ENRUTAR NUESTRA FUENTE desde el setup.bash de nuestro entorno de trabajo.

Hasta este punto, se han creado dos nodos: uno para **publicar** datos y otro para **suscribirse** a esos datos a través de un tópico. Estos nodos están diseñados para comunicarse entre sí, facilitando la interacción dentro del sistema de ROS2.

4. Servidor y cliente:

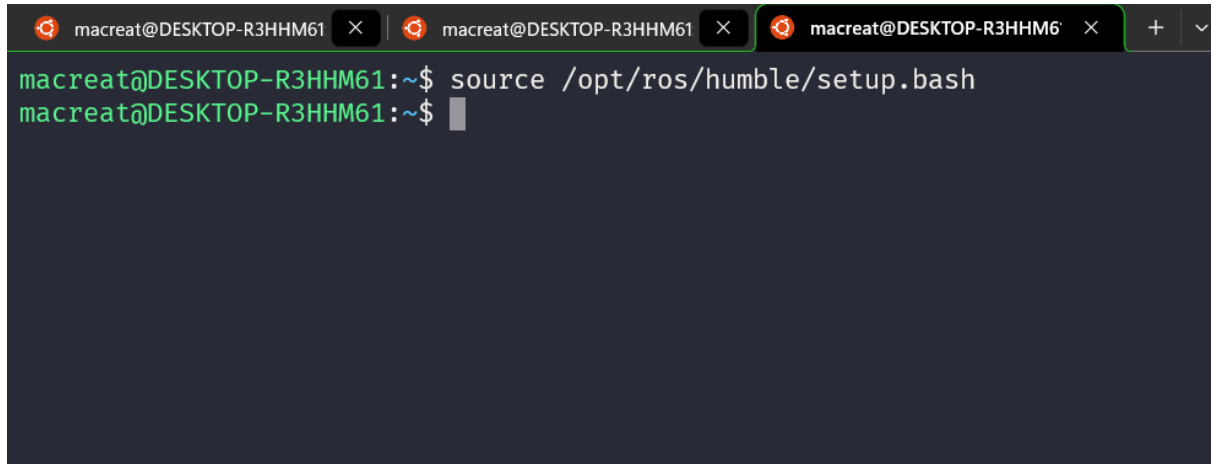
Creación de un Servicio y Cliente en Python en ROS2

Cuando los nodos se comunican mediante **servicios**, el nodo que envía una solicitud de datos se denomina **nodo cliente**, mientras que el nodo que responde a la solicitud se denomina **nodo servicio**. La estructura de la solicitud y la respuesta está determinada por un archivo **.srv**, que define los tipos de mensajes para la comunicación.

En el ejemplo utilizado, se crea un sistema simple de **adición de enteros**: un nodo solicita la suma de dos números enteros y el otro responde con el resultado.

1. Creación del paquete

- En una nueva terminal, primero **source** el entorno de ROS2 para asegurarse de que todo el entorno esté cargado correctamente. Esto se hace con el siguiente comando, que carga la configuración del entorno de **ROS2**:



```
macreat@DESKTOP-R3HHM61:~$ source /opt/ros/humble/setup.bash
macreat@DESKTOP-R3HHM61:~$
```

Luego, podemos crear un paquete desde mi directorio fuente (src)

Al ejecutar el comando para crear el paquete **py_srvcli**, la terminal devolverá un mensaje que verifica la creación del paquete y la estructura de todos los archivos y carpetas necesarios.

```
ros2_debs_backup ros2_packages_cache ros2_ws
macreat@DESKTOP-R3HHM61:~$ cd ros2_ws
macreat@DESKTOP-R3HHM61:~/ros2_ws$ ls
build install log src
macreat@DESKTOP-R3HHM61:~/ros2_ws$ cd src
bash: cd: src: No such file or directory
macreat@DESKTOP-R3HHM61:~/ros2_ws$ cd src
macreat@DESKTOP-R3HHM61:~/ros2_ws/src$ ls
..out myPyPackageDir py_pubsub ros_tutorials
macreat@DESKTOP-R3HHM61:~/ros2_ws/src$ ros2 pkg create --build-type ament_python --license Apache-2.0 py_srvcli --dependencies rclpy example_interfaces
going to create a new package
package name: py_srvcli
destination directory: /home/macreat/ros2_ws/src
package format: 3
version: 0.0.0
description: TODO: Package description
maintainer: ['macreat <macreat@todo.todo>']
licenses: ['Apache-2.0']
build type: ament_python
dependencies: ['rclpy', 'example_interfaces']
creating folder ./py_srvcli
creating ./py_srvcli/package.xml
creating source folder
creating folder ./py_srvcli/py_srvcli
creating ./py_srvcli/setup.py
creating ./py_srvcli/setup.cfg
creating folder ./py_srvcli/resource
creating ./py_srvcli/resource/py_srvcli
creating ./py_srvcli/py_srvcli/__init__.py
creating folder ./py_srvcli/test
creating ./py_srvcli/test/test_copyright.py
creating ./py_srvcli/test/test_flake8.py
creating ./py_srvcli/test/test_pep257.py
macreat@DESKTOP-R3HHM61:~/ros2_ws/src$ ls
..out myPyPackageDir py_pubsub py_srvcli ros_tutorials
macreat@DESKTOP-R3HHM61:~/ros2_ws/src$
```

Dependencias Automáticas en package.xml

El argumento **--dependencies** añadirá automáticamente las líneas de dependencias necesarias al archivo **package.xml**. Esto asegura que todas las dependencias requeridas para el correcto funcionamiento del paquete sean incluidas sin necesidad de intervención manual.

En este caso, se incluye la dependencia del paquete **example_interfaces**, que contiene el archivo **.srv** necesario para estructurar las solicitudes y respuestas del servicio. El archivo **.srv** define el tipo de mensaje que se utilizará en la comunicación entre el cliente y el servicio.

Estructura del archivo .srv

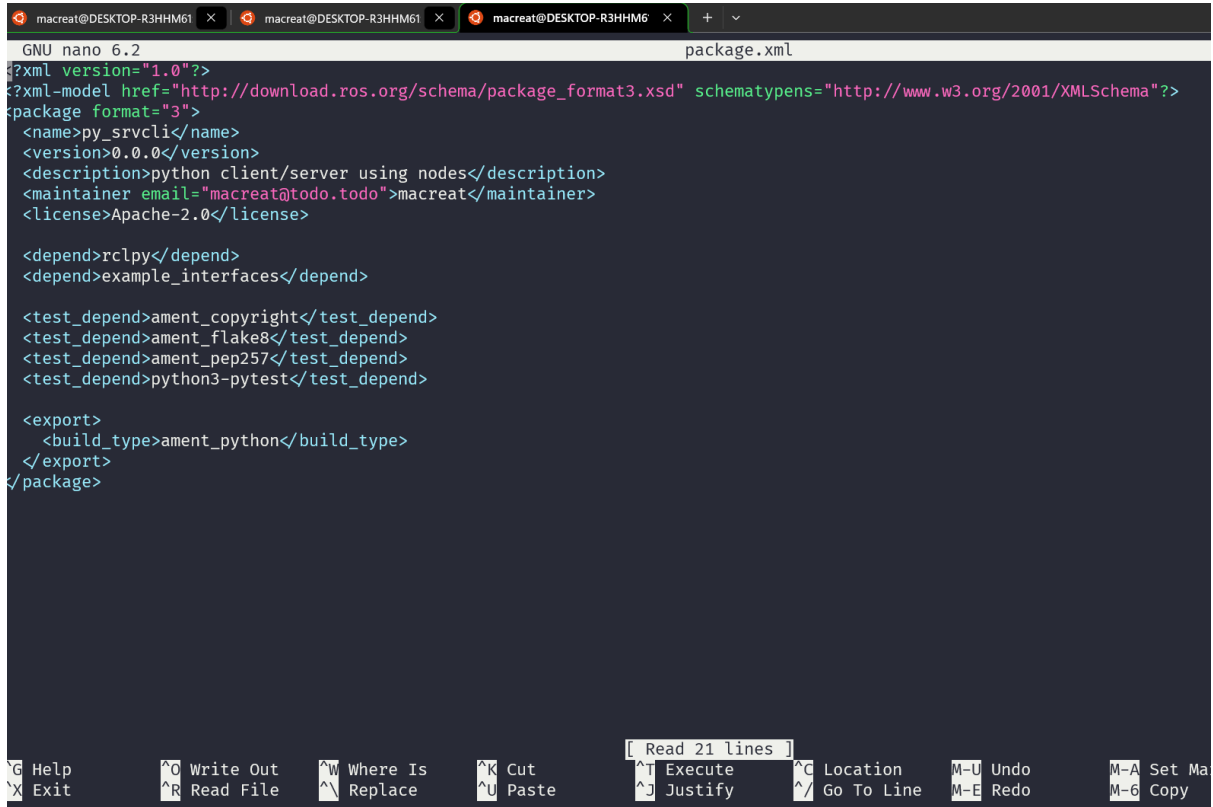
El archivo **.srv** utilizado en este ejemplo tendrá la siguiente estructura, que define los parámetros de entrada y salida para la operación de suma:

```
int64 a
int64 b
---
int64 sum
```

- **a y b:** Son los enteros que se sumarán.
- **sum:** Es el resultado de la suma, que será devuelto como respuesta.

1.1 Actualizar el archivo package.xml

Al utilizar el argumento **--dependencies** durante la creación del paquete, las dependencias necesarias se añaden automáticamente al archivo **package.xml**, lo que elimina la necesidad de hacerlo manualmente. Sin embargo, es importante completar el archivo **package.xml** con información adicional relevante, como la descripción del paquete, el correo electrónico y nombre del mantenedor, y la información de la licencia.



```
GNU nano 6.2 package.xml
<?xml version="1.0"?>
<?xml-model href="http://download.ros.org/schema/package_format3.xsd" schematypens="http://www.w3.org/2001/XMLSchema"?>
<package format="3">
  <name>py_srvcli</name>
  <version>0.0.0</version>
  <description>python client/server using nodes</description>
  <maintainer email="macreat@todo.todo">macreat</maintainer>
  <license>Apache-2.0</license>

  <depend>rclpy</depend>
  <depend>example_interfaces</depend>

  <test_depend>ament_copyright</test_depend>
  <test_depend>ament_flake8</test_depend>
  <test_depend>ament_pep257</test_depend>
  <test_depend>python3-pytest</test_depend>

  <export>
    <build_type>ament_python</build_type>
  </export>
</package>
```

Esto con el fin de que iguale a la configuración del archivo setup.py

1.2 Actualizamos el archivo:

```
GNU nano 6.2 setup.py
from setuptools import find_packages, setup

package_name = 'py_srvcli'

setup(
    name=package_name,
    version='0.0.0',
    packages=find_packages(exclude=['test']),
    data_files=[
        ('share/ament_index/resource_index/packages',
         ['resource/' + package_name]),
        ('share/' + package_name, ['package.xml']),
    ],
    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='macreat',
    maintainer_email='macreat@todo.todo',
    description='python client/server using nodes',
    license='Apache-2.0',
    tests_require=['pytest'],
    entry_points={
        'console_scripts': [
        ],
    },
)

Save modified buffer?
Y Yes
N No ^C Cancel
```

3. Creando el nodo del servicio:

En la carpeta de mi paquete, no el directorio de este, creamos un archivo llamado

Service_member_function.py con el siguiente contenido:

```
macreat@DESKTOP-R3HHM61 x macreat@DESKTOP-R3HHM61 x macreat@DESKTOP-R3HHM6 x + v
GNU nano 6.2 service_member_function.py
from example_interfaces.srv import AddTwoInts

import rclpy
from rclpy.node import Node

class MinimalService(Node):

    def __init__(self):
        super().__init__('minimal_service')
        self.srv = self.create_service(AddTwoInts, 'add_two_ints', self.add_two_ints_callback)

    def add_two_ints_callback(self, request, response):
        response.sum = request.a + request.b
        self.get_logger().info('Incoming request\na: %d b: %d' % (request.a, request.b))

        return response

def main():
    rclpy.init()

    minimal_service = MinimalService()

    rclpy.spin(minimal_service)

    rclpy.shutdown()

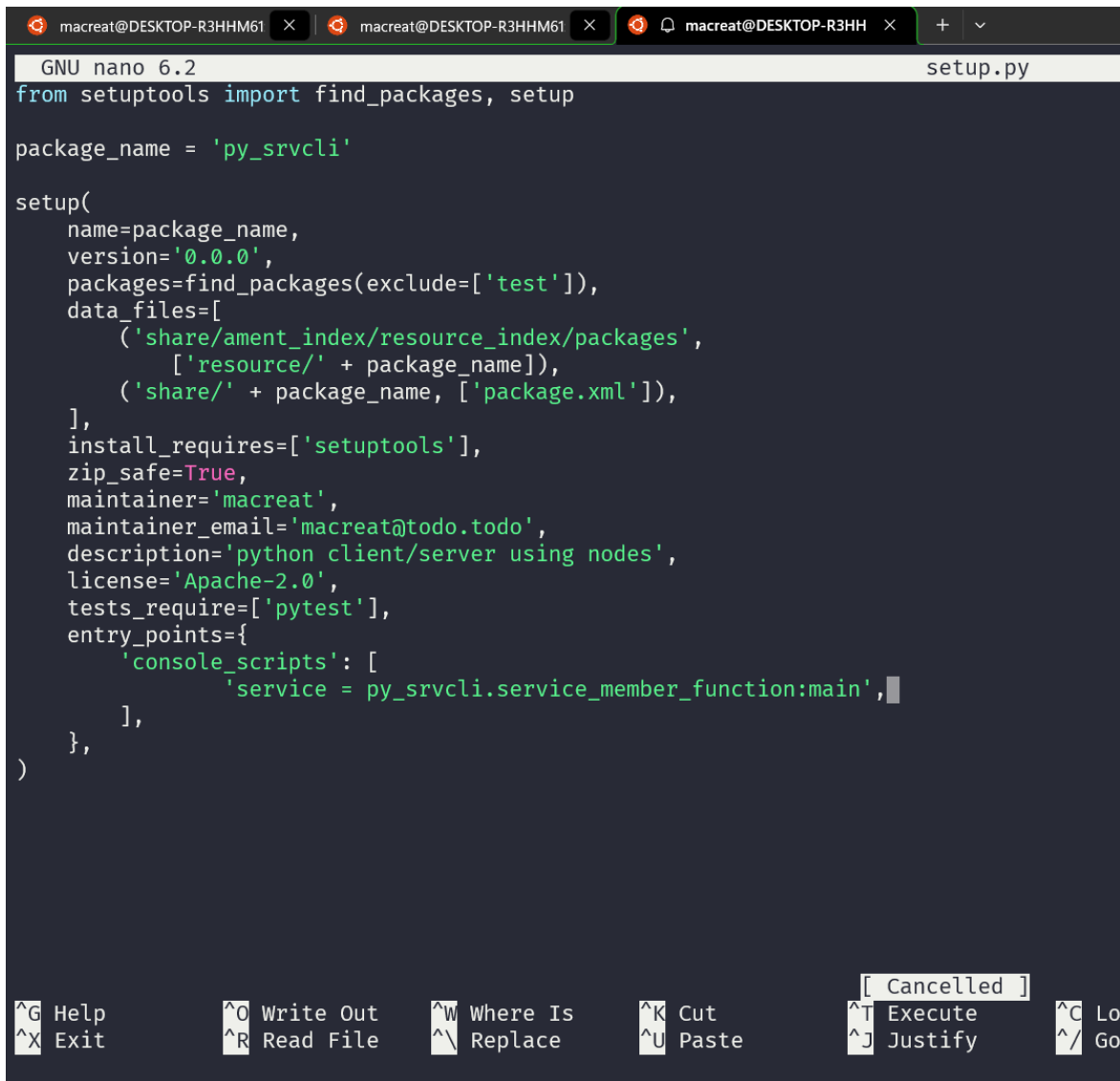
if __name__ == '__main__':
    main()
```

[Wrote 31 lines]

Help	Write Out	Where Is	Cut	Execute	Location
Exit	Read File	Replace	Paste	Justify	Go To Line

2.2 Ahora, procedemos añadiendo el punto de entrada en “entry_points”

Para permitir que el comando **ros2 run** funcione correctamente con el nodo que has creado, es necesario agregar el **punto de entrada** en el archivo **setup.py**. Este archivo se encuentra dentro del directorio de tu paquete, no dentro del subdirectorio **package**.



```
GNU nano 6.2                                setup.py
from setuptools import find_packages, setup

package_name = 'py_srvcli'

setup(
    name=package_name,
    version='0.0.0',
    packages=find_packages(exclude=['test']),
    data_files=[
        ('share/ament_index/resource_index/packages',
         ['resource/' + package_name]),
        ('share/' + package_name, ['package.xml']),
    ],
    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='macreat',
    maintainer_email='macreat@todo.todo',
    description='python client/server using nodes',
    license='Apache-2.0',
    tests_require=['pytest'],
    entry_points={
        'console_scripts': [
            'service = py_srvcli.service_member_function:main',
        ],
    },
)

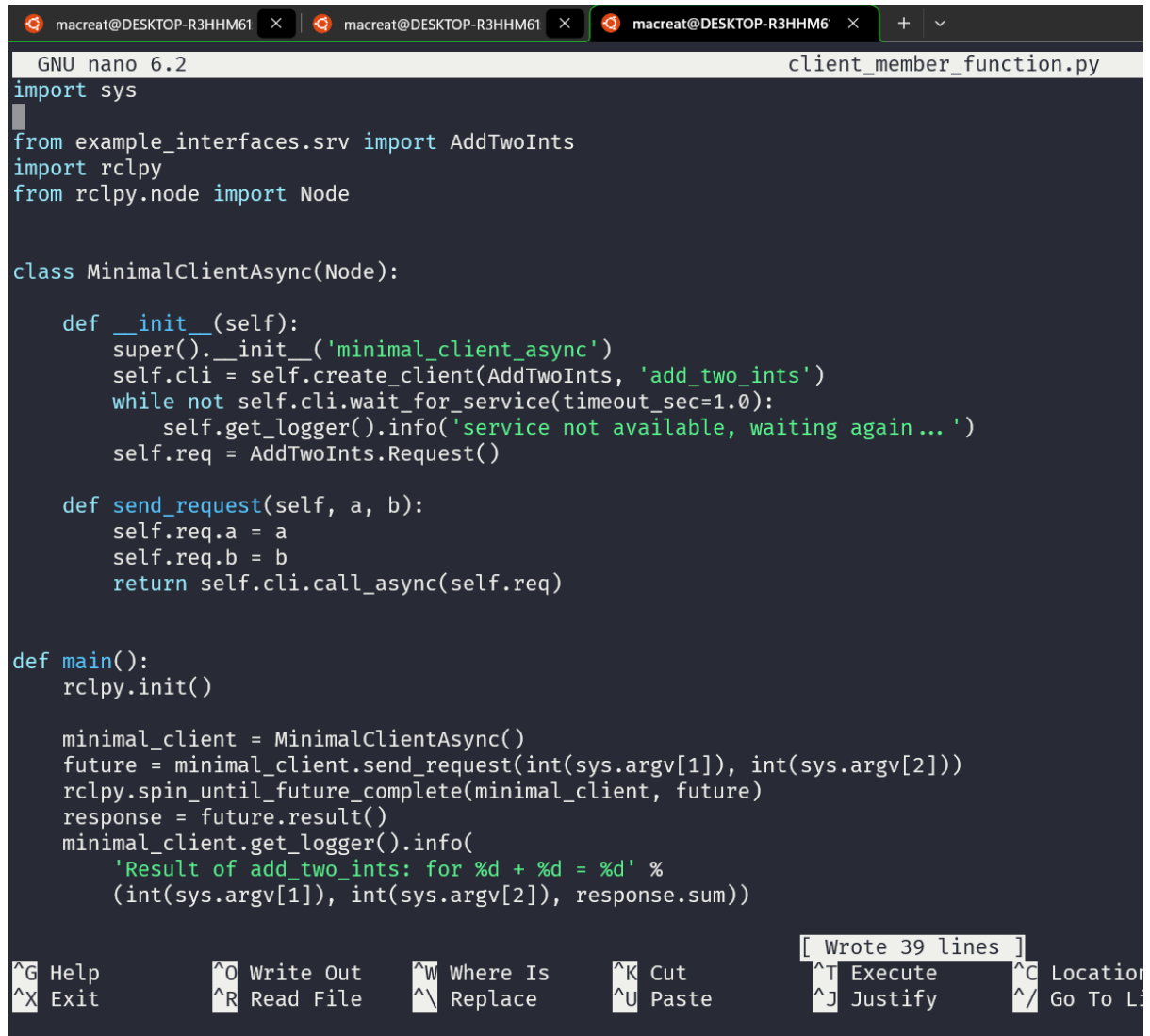
[ Cancelled ]
^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute  ^C Lo
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify  ^/ Go
```

Añadimos el servicio de mi paquete como archivo script de Python .py, este donde esta llamada mi función main ()

3. Procedemos creando el nodo del cliente.

En **ROS2**, la implementación de un nodo cliente para interactuar con un servicio se puede realizar mediante una **función miembro** (client_member function). Este tipo de función es útil para gestionar las solicitudes y respuestas dentro de un nodo cliente, específicamente cuando se interactúa con servicios.

Dentro de mi paquete, creamos el script `client_member_function.py`, con el siguiente contenido



```
GNU nano 6.2 client_member_function.py
import sys

from example_interfaces.srv import AddTwoInts
import rclpy
from rclpy.node import Node

class MinimalClientAsync(Node):

    def __init__(self):
        super().__init__('minimal_client_async')
        self.cli = self.create_client(AddTwoInts, 'add_two_ints')
        while not self.cli.wait_for_service(timeout_sec=1.0):
            self.get_logger().info('service not available, waiting again... ')
        self.req = AddTwoInts.Request()

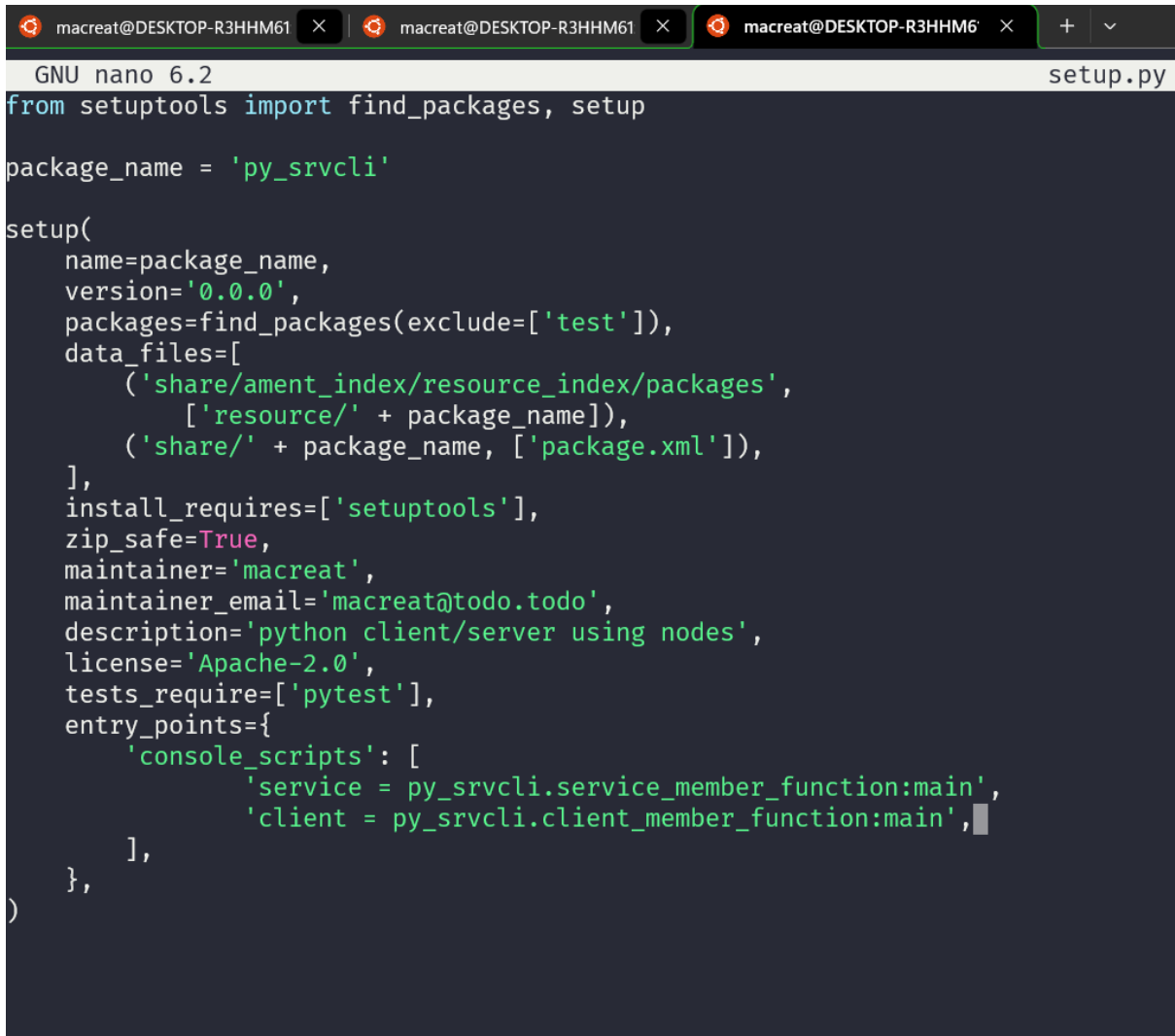
    def send_request(self, a, b):
        self.req.a = a
        self.req.b = b
        return self.cli.call_async(self.req)

def main():
    rclpy.init()

    minimal_client = MinimalClientAsync()
    future = minimal_client.send_request(int(sys.argv[1]), int(sys.argv[2]))
    rclpy.spin_until_future_complete(minimal_client, future)
    response = future.result()
    minimal_client.get_logger().info(
        'Result of add_two_ints: for %d + %d = %d' %
        (int(sys.argv[1]), int(sys.argv[2]), response.sum))

[ Wrote 39 lines ]
^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To L
```

3.2 También es importante asegurarnos de que añadamos el punto de entrada debajo de mi otro punto de entrada del servicio.



```
GNU nano 6.2                                     setup.py
from setuptools import find_packages, setup

package_name = 'py_srvcli'

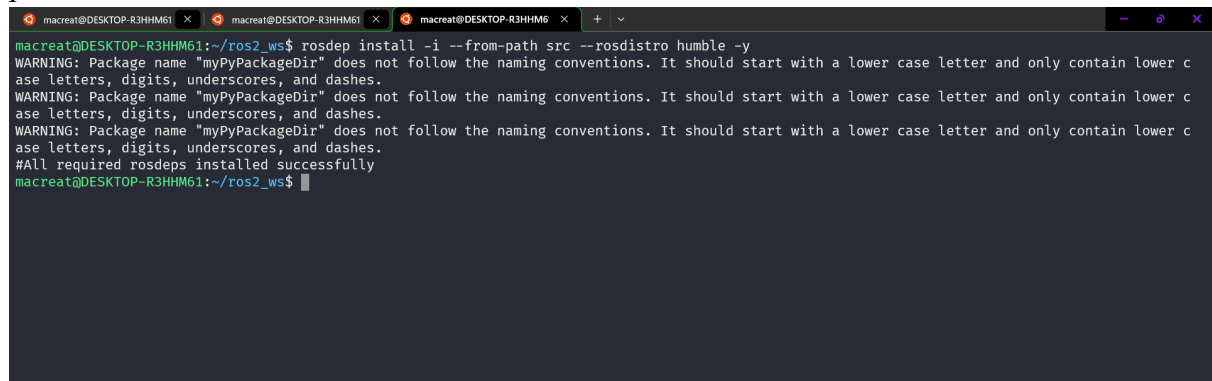
setup(
    name=package_name,
    version='0.0.0',
    packages=find_packages(exclude=['test']),
    data_files=[
        ('share/ament_index/resource_index/packages',
         ['resource/' + package_name]),
        ('share/' + package_name, ['package.xml']),
    ],
    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='macreat',
    maintainer_email='macreat@todo.todo',
    description='python client/server using nodes',
    license='Apache-2.0',
    tests_require=['pytest'],
    entry_points={
        'console_scripts': [
            'service = py_srvcli.service_member_function:main',
            'client = py_srvcli.client_member_function:main',
        ],
    },
)
```

4. Continuamos compilando y corriendo nuestro paquete en el entorno de trabajo:

Es una buena práctica ejecutar **rosdep** en la raíz de tu espacio de trabajo (**ros2_ws**) para verificar las dependencias faltantes antes de proceder con la construcción del paquete.

rosdep se encarga de instalar automáticamente las dependencias necesarias que no están

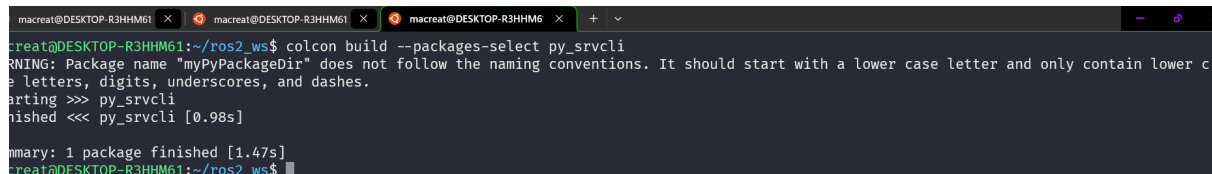
presentes en el sistema.



```
macreat@DESKTOP-R3HHM61:~/ros2_ws$ rosdep install -i --from-path src --rosdistro humble -y
WARNING: Package name "myPyPackageDir" does not follow the naming conventions. It should start with a lower case letter and only contain lower case letters, digits, underscores, and dashes.
WARNING: Package name "myPyPackageDir" does not follow the naming conventions. It should start with a lower case letter and only contain lower case letters, digits, underscores, and dashes.
WARNING: Package name "myPyPackageDir" does not follow the naming conventions. It should start with a lower case letter and only contain lower case letters, digits, underscores, and dashes.
#All required rosdeps installed successfully
macreat@DESKTOP-R3HHM61:~/ros2_ws$
```

Este comando verifica todas las dependencias necesarias para los paquetes en el espacio de trabajo y las instala automáticamente. Es importante recordar que **rosdep** debe ejecutarse en la raíz de tu espacio de trabajo para que pueda buscar correctamente todas las dependencias de los paquetes dentro de **src**.

También recordar que debemos compilar el paquete usando colcon:



```
macreat@DESKTOP-R3HHM61:~/ros2_ws$ colcon build --packages-select py_srvcli
WARNING: Package name "myPyPackageDir" does not follow the naming conventions. It should start with a lower case letter and only contain lower case letters, digits, underscores, and dashes.
Starting >>> py_srvcli
Finished <<< py_srvcli [0.98s]
Summary: 1 package finished [1.47s]
macreat@DESKTOP-R3HHM61:~/ros2_ws$
```

5. CORRER NUESTRO SERVICIO/CLIENTE

Ejecutar el Nodo del Servicio en un Nuevo Terminal

1. Abrir un Nuevo Terminal:

- Para trabajar en un nuevo terminal, primero abre una nueva ventana de terminal en tu sistema.

2. Navegar al Espacio de Trabajo:

- Luego, navega a la raíz de tu espacio de trabajo **ROS2** (ros2_ws)

Es necesario enrutar haciendo "source" al archivo de configuración de **ROS2** para cargar el entorno y las dependencias de tu espacio de trabajo. Esto se hace ejecutando el comando visto en pantalla.

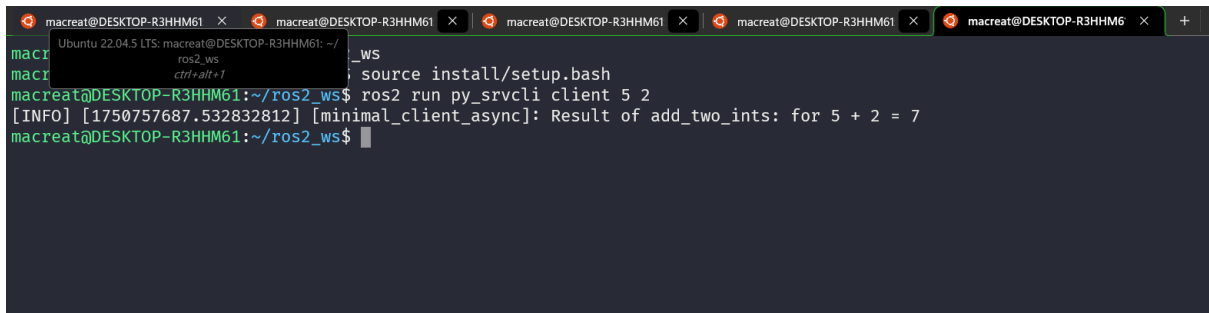
Después de realizar el sourcing, se procede a ejecutar el nodo del servicio, por ejemplo, el nodo servidor para manejar las solicitudes.



```
macreat@DESKTOP-R3HHM61:~$ cd ros2_ws
macreat@DESKTOP-R3HHM61:~/ros2_ws$ source install/setup.bash
macreat@DESKTOP-R3HHM61:~/ros2_ws$ ros2 run py_srvcli service
```

Hasta este punto, el nodo de servicio esperará una petición desde el nodo cliente:

Podemos abrir una terminal nueva, enrutar la fuente del espacio de trabajo, y luego correr el nodo del cliente seguido de los números enteros que querramos para realizar la petición (que en este caso es una suma)



```
macreat@DESKTOP-R3HHM61:~$ cd ros2_ws
macreat@DESKTOP-R3HHM61:~/ros2_ws$ source install/setup.bash
macreat@DESKTOP-R3HHM61:~/ros2_ws$ ros2 run py_srvcli client 5 2
[INFO] [1750757687.532832812] [minimal_client_async]: Result of add_two_ints: for 5 + 2 = 7
macreat@DESKTOP-R3HHM61:~/ros2_ws$
```

Ahora, podemos observar nuestro nodo de SERVICIO:

```
macreat@DESKTOP-R3HHM61:~$ cd ros2_ws
macreat@DESKTOP-R3HHM61:~/ros2_ws$ source install/setup.bash
macreat@DESKTOP-R3HHM61:~/ros2_ws$ ros2 run py_srvcli service

[INFO] [1750757687.520586843] [minimal_service]: Incoming request
a: 5 b: 2
```

Notemos, como mi nodo cliente publica el resultado :

```
macreat@DESKTOP-R3HHM61:~$ cd ros2_ws
macreat@DESKTOP-R3HHM61:~/ros2_ws$ source install/setup.bash
macreat@DESKTOP-R3HHM61:~/ros2_ws$ ros2 run py_srvcli client 5 2
[INFO] [1750757687.532832812] [minimal_client_async]: Result of add_two_ints: for 5 + 2 = 7
macreat@DESKTOP-R3HHM61:~/ros2_ws$
```

Esto verifica el procedimiento realizado también en el tutorial de servicio y cliente usando nodos en ROS2.

ROS2 en el proyecto ‘robot autónomo’

- El IMU (**unidad inercial**) publica orientación y aceleración.
- Un nodo de navegación **suscribe esa información** para estimar posición o detectar movimientos.

- Puedes tener otro nodo que publique comandos de velocidad (/cmd_vel) y un nodo controlador que los escuche para mover las ruedas Mecanum.

2. CLIENT /SERVICE

ESTE PATRON ES REQUERIDO CUANDO SE NECESITA UNA ACCION PUNTUAL

Cliente	Servidor (Service)	Ejemplo robot
Hace una solicitud	Ejecuta la acción y devuelve una respuesta	"Gira 90°" → el robot lo hace y responde "OK"

Componentes a usar:

Componente	Rol ROS2	Tipo de comunicación
IMU (unidad inercial)	Publicador de orientación/velocidad	publisher
Encoders / motores	Publicadores de odometría y receptores de comandos	pub/sub
Nodo de control (embebido en RPi)	Suscriptor de datos / Publicador de comandos	pub/sub
Nodo de navegación (SLAM o Path planning)	Cliente de servicios / Subscriber de mapa	services y subscribers
Interfaz de usuario web	Cliente que envía comandos por ROS Bridge	service client
Simulación (Rviz, Gazebo)	Visualización de sensores y estado	pub/sub

4. Conclusiones

- La instalación de **ROS2 Humble** sobre **Ubuntu 22.04 en WSL** estableció un entorno robusto para el desarrollo de aplicaciones robóticas, facilitando la ejecución de tutoriales y simulaciones sin inconvenientes y la gestión de tareas en entornos robóticos complejos.
 - A través de los tutoriales de **CLI Tools** y **Client Libraries**, se aprendió a gestionar nodos, tópicos, servicios y parámetros desde la terminal, experimentando con la comunicación entre nodos mediante **publicadores** y **suscriptores** en **Python**.
 - Las herramientas como **Turtlesim** y **rqt** permitieron simular y visualizar interacciones de nodos en tiempo real, validando la funcionalidad de ROS2 y reforzando la comprensión sobre la gestión de tareas en entornos robóticos complejos.
-

5. Referencia

- ROS2 sitio oficial: <https://docs.ros.org/en/humble/index.html>
- ROS2 tutoriales: <https://docs.ros.org/en/humble/Tutorials.html>